TOOFAB: User's Manual

Francis le Roux
f.a.l.le.roux@tue.nl
Zuan Khalik
Version 1.5.5

Abstract

This document describes the features and the use of the TOOlbox for FAst Battery simulation (TOOFAB). TOOFAB is a battery simulation toolbox that can accurately and efficiently simulate the Doyle-Fuller-Newman model, which is an electrochemical battery model that describes the internal electrochemical processes in a battery. This document provides a detailed description of the simulation procedure implemented in `DFN.m` and a reference of every internal (local) function that the toolbox uses, so that users who want to understand, extend, or debug the toolbox can see exactly how the pieces fit together. It also documents `parameter_determination.m`, which estimates DFN model parameters from experimental current, voltage and/or temperature data.

Contents

1 Before you begin **3**

1.1 What is TOOFAB? 3

1.2 Prerequisites 4

1.3 Installing the Toolbox 4

2 Using the Toolbox **4**

2.1 Running Simulations 4

2.2 Choosing Simplifications 4

2.3 Choosing Grid Parameters 5

2.4 Ageing and Thermal Dynamics 5

3 Example **5**

4 Functions **6**

4.1 DFN 6

4.2 Simulation Procedure: How `DFN.m` Works Step by Step . . 10

4.3 Description of Internal (Local) Functions 12

5 User-Configurable Parameters **15**

5.1 Simulation Parameters 15

5.2 Material-Specific Parameters 17

6 Parameter Determination **20**

6.1 Overview 20

6.2 parameter_determination 21

6.3 Example 23

6.4 Procedure: How `parameter_determination.m` Works . . 23

6.5 Description of Internal (Local) Functions 24

1 Before you begin

1.1 What is TOOFAB?

The main feature of TOOFAB (TOOlbox for FAst Battery simulation) is an accurate and computationally efficient simulation of the Doyle-Fuller-Newman (DFN) model [1] based on the implementation and results presented in [2]. The DFN model is an electrochemistry-based battery model that describes the internal electrochemical processes (i.e., the concentration and potential gradients). This model has been used in a variety of applications, such as battery cell design and battery-management-system functionalities such as state-of-charge estimation, state-of-health estimation and fast charging.

While there are already several battery simulation toolboxes (freely) available that solve the DFN model equations, e.g. [3, 4, 5], our developed toolbox provides several advantages compared to the currently available toolboxes. Firstly, with the implementation used that is presented in [2], the computation times are generally significantly smaller than the currently available toolboxes, as is shown in [2]. Secondly, because the model equations have been solved directly, without the use of any other software or MATLAB toolboxes, the developed battery simulation toolbox does not require installing any other toolboxes. Finally, for the same reason as given in the previous point, i.e., that the model equations are solved directly without any other toolboxes, the MATLAB code is fairly easily translatable into C-code, which can either be made into an executable for even faster simulation, or can be used to implement the model on an embedded system, which are generally programmed in C.

The complete feature list of TOOFAB is as follows:

- Accurate and computationally efficient simulation of the DFN model describing the internal electrochemical processes of a battery.
- Ability to easily use and change concentration- and temperature-dependent parameters.
- Ability to easily configure the toolbox for a desired trade-off between accuracy and computational efficiency through selection of model simplifications and adjusting the coarseness of the discretization grid.
- Option to use a battery ageing model describing capacity fade due to SEI (solid-electrolyte interphase) formation, lithium plating, and positive-electrode (cathode) oxidation side reactions [6].
- Option to use a lumped thermal model [7] describing the thermal dynamics.
- Ability to efficiently use the model in a closed-loop setting, e.g., to simulate a controller acting on the battery.
- Ability to estimate DFN model parameters from experimental current/voltage(/temperature) data, including a sensitivity analysis to rank parameters by identifiability and a bounded nonlinear least-squares fit of the most influential subset (Section 6).

1.2 Prerequisites

This toolbox only requires a working version of MATLAB. The toolbox has been tested with MATLAB R2023a, but should work with any MATLAB version equal to or newer than MATLAB R2023a. This compatibility requirement comes from the feature that allows local functions, added to MATLAB since version R2016b.

1.3 Installing the Toolbox

There are two options for installing the toolbox. The first option is to download the toolbox manually from the project's GitHub repository. Alternatively, users can clone the repository directly using Git.

- **Manual download:** Download the toolbox from the GitHub repository and extract it to a desired location.
- **Git clone:** Clone the repository using Git with the following command:

```
git clone https://github.com/tue-battery/TOOFAB.git
```

Once the toolbox has been downloaded, there are two ways to make the `DFN.m` function available in MATLAB. The entire toolbox can be added to the MATLAB path, which makes all of its functions available regardless of the current working directory. This can be done by executing the following commands from within the toolbox directory:

```
addpath(genpath(pwd))
savepath
```

Alternatively, if only the `DFN.m` function is required, the `DFN.m` file can simply be copied to the folder in which the user is working. MATLAB will then be able to access the function directly from the current working directory.

2 Using the Toolbox

2.1 Running Simulations

Simulations can be run using the `DFN` function. The various arguments of `DFN` are explained in Section 4.1. For example, if the desire is to simulate the battery with the default parameters with a constant current of 30 A, for a duration of 2000 seconds with 0% state-of-charge as the initial condition, then `DFN` can be called as follows:

```
out = DFN(30,2000,0)
```

In Section 3, more examples can be found of how `DFN` can be used.

2.2 Choosing Simplifications

The model simplifications as considered in [2] can be chosen using either `simp` or `set_simp`. The main conclusions on the simplifications made in [2] are as follows

- Simplification [S1] linearizes the Butler-Volmer equation describing the reaction rate. This simplification generally does not lead to a significant sacrifice in accuracy.
- Simplification [S2] relates to the simplification of concentration-dependent parameters (`kappa`, `De`, `dlnfdce` and `Ds_pos`). Simplification [S2-I] assumes that there is a 1 step delay in how the concentration affects the parameter, i.e., the parameter is evaluated once per time step using the concentration at the previous time step, rather than being re-evaluated at every Newton iteration. This simplification has a negligible effect on accuracy. Simplification [S2-II] assumes the given concentration-dependent parameters to be constant (evaluated once, at the initial concentration, for the entire simulation). This simplification does not lead to a significant impact on accuracy in most cases.
- Finally, Simplification [S3], approximates diffusion along the radial direction in the particles with a 2-parameter polynomial approximation instead of the full finite-difference discretization. This simplification may sometimes lead to a significant loss in accuracy, but substantially reduces the number of states.

Each of the four concentration-dependent parameters can be simplified independently; `set_simp` is a compact vector form of `simp`, see Section 5 for the precise encoding.

2.3 Choosing Grid Parameters

The grid parameters can be chosen either through `grid` or `set_grid`. The main conclusions on the selection of the grid parameters made in [2] are as follows.

- For a high-energy cell, generally n_n, n_s, n_p have to be chosen relatively large, as diffusion throughout the cell is generally more restricted in a high-energy cell. On the other hand, $n_{r,n}$ and $n_{r,p}$ can be chosen relatively small, as diffusion throughout the particles is less restrictive.
- For the considered high-power cell, the opposite was observed. Here, n_n, n_s, n_p were chosen relatively small, while $n_{r,n}$, $n_{r,p}$ was chosen relatively large.

2.4 Ageing and Thermal Dynamics

In addition to the isothermal DFN model, TOOFAB can optionally simulate the lumped thermal dynamics of the cell (Section 5) and a battery ageing model that accounts for three capacity/power fade mechanisms: SEI formation, lithium plating, and positive-electrode oxidation (Section 5). Both are disabled by default and can be enabled independently through the parameters `thermal_dynamics` and `ageing`. The internal bookkeeping required to run these extensions is described together with the rest of the time-stepping procedure in Section 4.2.

3 Example

The `Examples` folder contains several scripts that illustrate typical use cases of the toolbox.

`Basic_DFN_example.mlx` : A MATLAB live script that walks through a basic DFN simulation interactively, including the use of `BPX_to_TOOFAB.m`, a function that converts a Battery Parameter eXchange (BPX) JSON parameter set into a TOOFAB-compatible parameter struct.

4 Functions

4.1 DFN

`DFN`: The `DFN` function is the main, user-facing modeling function. Sections 4.2 and 4.3 describe, respectively, what `DFN.m` does step by step and what each of its internal (local) functions does; the intent of those two sections is to let a reader connect a specific line of code, or a specific local function, to its role in the overall algorithm.

4.1.1 Syntax

```
out = DFN(input_current,tf,init_cond,param)
```

4.1.2 Input Arguments

`input_current`: This field contains information about the current profile. This field can be provided in one of three forms:

- As a **scalar** value. The value is used as the constant applied current from time `0` until time `tf`.
- As an **array**. The first column of the array must specify the time samples and the second column must specify the current levels at each specified time sample. An optional third column can be provided to additionally specify a temperature profile at each time sample; if given, the cell temperature is interpolated directly from this column instead of being computed from the thermal model. How the current (and temperature, if given) is interpolated and extrapolated in between the time samples can be specified with the parameters `current_interp` and `current_extrap`, respectively.
- As a **function handle**, which points to a function that takes the applied current, terminal voltage, state-of-charge, concentrations and potentials, a user memory struct, and the parameter struct as input, and mainly provides the next applied current as output. This form is especially useful when the battery is desired to be controlled in closed-loop (see `Basic_DFN_example.m`).

When provided as a function handle, the function that it points to must have the following input arguments, in the following specific order

1. `k` [-] is the current iteration (i.e., time step index) of the main time integration loop.
2. `t` [s] is the current simulation time.
3. `i_app` [A] is the complete vector of the applied current up to the current time step.
4. `V` [V] is the complete terminal voltage vector up to the current time step.
5. `soc` [-] is the complete state-of-charge vector up to the current time step.

6. `cs` [mol/m^3] is the complete solid-phase concentration matrix up to the current time step.
7. `ce` [mol/m^3] is the complete electrolyte-phase concentration matrix up to the current time step.
8. `phis` [V] is the complete solid-phase potential matrix up to the current time step.
9. `phie` [V] is the complete electrolyte-phase potential matrix up to the current time step.
10. `mem` is a MATLAB `struct` that can be used to store memory variables to be used throughout the simulation (e.g., controller states, or flags indicating which phase of a cycle is active).
11. `p` is a MATLAB `struct` containing all the defined parameter fields (i.e., `param`, after being merged with the default parameters).

Furthermore, the function that the function handle points to must have the following output arguments, in the following specific order

1. `i_app_next` [A] is the applied current at the next time step
2. `mem` is a necessary output argument to ensure that the memory functionality works. The contents of `mem` can be accessed after the simulation through `out.mem`.
3. `end_simulation` is a flag that can be set to 1 to stop the simulation after the current time step. Set this flag to 0 if its use is not desired.
4. `dt` [s] is an optional fourth output argument that specifies the step size to use for the next time step (if desired).

Note that changing the step size requires the re-computation of the time-invariant system matrices (`fcn_system_matrices`, Section 4.3), so changing the step size often throughout the simulation is not recommended for computational efficiency.

`tf`: Final integration time, in seconds.

`init_cond`: This field specifies the initial conditions of the cell. This field can be specified either as a scalar (or two-element vector), or as a MATLAB `struct` containing information on the desired initial conditions.

When specified as a **scalar**, the toolbox assumes steady-state conditions. Then, the value can be either chosen to represent an initial voltage or as an initial state-of-charge. To specify an initial state-of-charge, a value between 0 and 1 must be chosen. When a value larger than 2 is given for `init_cond`, the toolbox instead interprets `init_cond` as an initial open-circuit voltage and inverts the equilibrium-potential curve to find the corresponding state-of-charge.

When specified as a **two-element vector** `[init_cond(1) init_cond(2)]`, the first element is interpreted as above (SOC or voltage), and the toolbox additionally enables the ageing model (equivalent to setting `param.ageing = 1`) and interprets the second element as the battery's already-aged capacity, in Coulomb, at the start of the simulation. Internally, the toolbox inverts the capacity-fade relationship (function `fcn_Q_inv`, Section 4.3) to determine the equivalent cumulative charge loss and stoichiometric window that is consistent with the specified aged capacity.

When `init_cond` is given by a `struct`, the toolbox assumes non-steady-state (e.g., a previously simulated) initial condition. The struct must contain the following required fields

- `cs` [mol/m^3] is the solid-phase concentration vector at the initial time step
- `ce` [mol/m^3] is the electrolyte-phase concentration vector at the initial time step
- `phis` [V] is the solid-phase potential vector at the initial time step

Additionally, if thermal dynamics are enabled, i.e. the parameter `thermal_dynamics` is set to 1, the following field must also be specified

- `T` [K] is the temperature at the initial time step

Additionally, if ageing is enabled, i.e., the parameter `ageing` is set to 1, the following fields must also be specified

- `Closs` [C] is the amount of charge lost due to Li-ion loss at the initial time step.
- `Rf` [Ω/m^2] is the SEI film resistance at every node at the initial time step.

In practice, a struct of exactly this form is returned as `out.states_end` at the end of a simulation (Section 4.1.3), which makes it convenient to continue a simulation, or a series of simulations (e.g. one ageing cycle at a time, as in `ageing_example.m`), from where a previous one left off.

`param`: This is an optional input argument, given as a `struct`. Here, any desired non-default parameter fields of the configurable parameters given in Section 5 can be specified. Configurable parameters that have not been specified will be assumed a certain default value.

4.1.3 Output Arguments

`out`: is a `struct` that contains all the output variables. The variables that are contained in `out` are

- `t` [s] is the simulation time vector. The values at the column indices of the time-varying outputs below correspond to the times given in `t`.
- `cs` [mol/m^3] is the solid-phase concentration. The values of `cs` at every time step are given by the rows of `cs`. The values of the concentration in the negative electrode are given from row index 1 through `n_n*n_rn`. The concentration values along the radial direction in the negative electrode are given by every `n_rn` rows for every of the `n_n` particles. The values of the concentration in the positive electrode are given from row index `n_n*n_rn+1` to the final row index. The concentration values along the radial direction in the positive electrode are given by every `n_rp` rows for every of the `n_p` particles.
- `ce` [mol/m^3] is the electrolyte-phase concentration at every discretization node across the cell width, at every time step.
- `phis` [V] is the solid-phase potential across the cell width. Rows corresponding to the separator are filled with `NaN`, since the solid-phase potential is not defined there.

- `phie` [V] is the electrolyte-phase potential across the cell width (including the separator).
- `cs_bar` [mol/m^3] is the solid-phase concentration *at the surface of the particles*, across the cell width (separator rows filled with `NaN`).
- `stoich` [-] is the stoichiometry (i.e., `cs_bar` normalized by `cs_max_neg`/`cs_max_pos`) at the surface of the particles, across the cell width.
- `jn` [mol/m^2/s] is the net molar flux (pore-wall flux) exiting the particles, across the cell width.
- `U` [V] is the equilibrium (open-circuit) potential at every node across the cell width.
- `eta` [V] is the main-reaction (intercalation) over-potential at every node across the cell width.
- `V` [V] is the terminal voltage.
- `i_app` [A] is the applied current.
- `T` [K] is the cell temperature.
- `soc` [-] is the cell state-of-charge.
- `sim_time` [s] is the wall-clock time taken to run the simulation, useful for benchmarking.
- `param` is the `struct` containing the configuration of the configurable parameters that have been used for the simulation. The fields in `param` are explained in Section 5.
- `mem` is the memory struct used when providing a function handle to `input_current`, as it stood at the end of the simulation.
- `states_end` is a `struct` containing the final value of the states that need to be specified through `init_cond` in order to continue the simulation: `phis`, `cs`, `ce`, and `T` (if thermal dynamics are enabled), plus `Closs` and `Rf` if ageing is enabled. See the discussion of struct-valued `init_cond` above.
- `stoichs` is a cell array recording the evolution of the stoichiometric window (`s0_neg`, `s100_neg`, `s0_pos`, `s100_pos`) over the course of the simulation, updated every 500 time steps. Only output when `ageing` is set to 1.
- `ageing` is a `struct` that is only output when parameter `ageing` is set to 1. It contains the following fields:
  - `Cl_ox`, `Closs_SEI`, `Closs_LPL` [C] are the cumulative charge lost due to, respectively, the cathode-oxidation, SEI-formation, and lithium-plating side reactions.
  - `Closs` [C] is the cumulative charge lost due to all enabled side reactions combined (i.e., the sum of the three fields above). Note that the fresh capacity of the cell minus `Closs` does not equal the aged capacity of the cell directly; the relation between the two is given by the capacity-fade model (`fcn_Q`, Section 4.3).
  - `s0_neg_aged`, `s100_neg_aged`, `s0_pos_aged`, `s100_pos_aged` [-] are the aged stoichiometric values at 0%/100% SoC in the negative/positive electrode at the end of the simulation.

- `Rf` [Ω/m^2] is the SEI film resistance at every node, at every time step.
- `jsei` [$mol/m^2/s$] is the SEI-formation side-reaction molar flux.
- `jlpl` [$mol/m^2/s$] is the lithium-plating side-reaction molar flux.
- `SEIFac` [-] is the resistance-limiting factor applied to the SEI-formation flux when `ageingMech.LimitedSEI` is enabled.
- `eta2`, `eta_ox` [V] are the over-potentials driving the SEI-formation and oxidation side reactions, respectively.
- `epse_neg` [-] is the evolution of the electrolyte volume fraction in the negative electrode (porosity loss due to SEI/lithium-plating deposits).
- `epss_pos` [-] is the evolution of the active-material volume fraction in the positive electrode (loss due to cathode oxidation).
- `LPLConc` [-] is an internal state tracking the lithium-plating deposit inventory that is available for stripping (used in the exponential term of `jlpl`).
- `j_ox` [$mol/m^2/s$] is the cathode-oxidation side-reaction molar flux. Only output when `ageing` is set to 1.

4.2 Simulation Procedure: How `DFN.m` Works Step by Step

This section describes, in order, what `DFN.m` actually does between being called and returning `out`. The local functions named in each step are documented individually in Section 4.3.

1. **Default parameters.** `p = default_parameters()` builds the struct `p` with every configurable parameter set to its default value (Section 5).
2. **User parameters.** If a `param` struct was supplied, `p = process_param(p, param)` overwrites the relevant defaults with the user-supplied values, and resolves the compact `set_simp`/`set_grid` vector forms against the more explicit `simp`/`grid` struct forms.
3. **Ageing shorthand.** If `init_cond` was supplied as a vector with more than one element, `p.ageing` is forced to 1.
4. **Parse `input_current`.** Depending on whether `input_current` is a function handle, a two- or three-column array, or a scalar, the toolbox sets an internal `input_mode` flag (2, 1, or 0 respectively) that controls how the applied current (and, for the three-column case, the temperature) is looked up at every time step. For array input, a `griddedInterpolant` is built once, upfront.
5. **Expand parameters onto the grid.** `fcn_system_vectors` converts the (mostly scalar) user-facing parameters into per-node vectors over the spatial and radial discretization grids, computes derived quantities such as `cs_max_neg`/`cs_max_pos`, and calls `deter_equil` to build the equilibrium-potential interpolants and their derivatives. If ageing is enabled, the stoichiometric window and the aged capacity `p.Cbat` consistent with the specified initial charge loss are also computed here (via `fcn_Q`).

6. **Build the time-invariant system matrices.** `fcn_system_matrices` assembles the sparse matrices of the semi-analytical discretization that do not change within a time step (e.g. the solid- and electrolyte-phase diffusion operators, the solid-phase potential operator, and, depending on the chosen simplifications, the concentration-dependent transport-parameter matrices via `De_matrices`, `Ds_matrices`, `kappa_matrices` and `nu_matrices`).

7. **Initial state.** `init` sets the state at $t = 0$, either from the steady-state SOC/voltage assumption or by copying the fields of a supplied `init_cond` struct. The initial state-of-charge is computed with `compute_soc` and checked to lie within $[0, 1]$ (a warning is issued otherwise).

8. **Pre-allocate outputs** for `cs`, `ce`, `phis`, `phie`, `V`, `T`, and the various ageing-related flux/state arrays, sized according to `tf` and the (initial) step size `p.dt`.

9. **Time-stepping loop** (`while not(end_simulation)`), for each time step $t \to t + 1$:

   (a) *Determine the applied current and time step.* If `input_mode` is 2 (function handle), the user function is called to obtain the next current, the updated `mem` struct, the `end_simulation` flag and, optionally, the next step size `dt`. If the previous time step's Newton iteration failed to converge (see below), the step size is instead reduced automatically. If the step size changed since the previous time step, the time-invariant system matrices are rebuilt via `fcn_system_matrices`.

   (b) *Update matrices that depend on the previous time step's concentration*, via `fcn_system_matrices2`, for any of the four concentration-dependent transport parameters using simplification [S2-I] (which are evaluated once per time step rather than at every Newton iteration).

   (c) *Solve the coupled algebraic equations with Newton's method.* For up to `p.iter_max` iterations: `fcn_F` evaluates the residual $F(\phi_s)$ together with all quantities that follow algebraically from a candidate solid-phase potential ϕ_s (the pore-wall flux `jn`, electrolyte concentration `ce` and potential `phie`, the ageing side-reaction fluxes `jsei`/`jlpl`/`j_ox`, the solid-phase concentration `cs`, the exchange current density `i0`, the equilibrium potential `U`, and the over-potential `eta`); `fcn_J` evaluates the Jacobian of F with respect to ϕ_s; and ϕ_s is updated with a Newton step, $\phi_s \leftarrow \phi_s - J^{-1}F$. The loop stops early once $\|F\|_2 <$ `p.tol`. If `p.iter_max` is reached without convergence, the step is flagged as failed, the time step is halved (or reduced much more aggressively after repeated failures) and the same time step is retried.

   (d) *Terminal voltage.* $V(t{+}1)$ is computed from the boundary values of ϕ_s, the applied current, and the current-collector resistance `R_cc`.

   (e) *Ageing update* (only if `p.ageing`). The cumulative charge-loss integrals (`Closs_SEI`, `Closs_LPL`, `Closs_ox`) are advanced by one step using the corresponding fluxes from `fcn_F`; the SEI film resistance `Rf` and the negative-electrode porosity

epse_neg are updated from the SEI-formation and lithium-plating fluxes; the positive-electrode active-material volume fraction epss_pos is updated from the oxidation flux. Every 500 time steps, fcn_Q recomputes the aged capacity p.Cbat and the stoichiometric window (s0_neg, s100_neg, s0_pos, s100_pos) from the cumulative charge loss so far, and the transport-property vectors that depend on porosity/volume fraction (sigma_eff, a_s, eps_s) are refreshed accordingly.

(f) *State-of-charge*, via compute_soc.

(g) *Thermal update.* If p.thermal_dynamics is enabled, fcn_T advances the lumped thermal energy balance by one step using the reaction over-potentials and heat-generation terms. If a temperature profile was instead supplied as a third column of input_current, the temperature is looked up from that interpolant. Otherwise the temperature is held at the (possibly time-varying, via mem.T_amb) ambient temperature.

(h) *Termination checks.* The simulation stops if: the time step becomes numerically negligible without the Newton iteration converging (the model has likely been pushed into an infeasible region); the solid- or electrolyte-phase stoichiometry/concentration leaves its physical bounds; the terminal voltage leaves [Vmin, Vmax]; the Newton iteration has repeatedly failed to converge even after reducing the step size; the simulation time is not progressing; or the final time tf has been reached.

10. **Assemble the output struct.** Once the loop ends, the pre-allocated arrays are trimmed to the actual number of time steps taken, reshaped to include NaN rows for the separator where appropriate, and packaged into out, together with out.ageing (if applicable) and out.states_end (Section 4.1.3).

4.3 Description of Internal (Local) Functions

All functions described in this section are local functions defined within DFN.m; they are not intended to be called directly by the user, but understanding what they do is useful for anyone wanting to extend the model (e.g. with a new side-reaction or a new simplification) or to debug an unexpected result. Functions are listed in roughly the order in which they first become relevant during a simulation (matching Section 4.2).

default_parameters(): Returns the struct p populated with every configurable parameter at its default value: simulation settings (step size, grid, tolerances, voltage bounds, verbosity), physical constants, and the default DFN, thermal, and ageing model parameters. This is always the first function called by DFN.

process_param(p, param): Overwrites the fields of the default parameter struct p with any fields present in the user-supplied param struct. Also determines whether the user specified an EMF-based equilibrium potential (setting p.equil_mode), and reconciles the struct-based (simp, grid) and compact vector-based (set_simp, set_grid) ways of specifying simplifications and grid sizes, with the vector form taking precedence when both are given.

fcn_system_vectors(p, init_cond, auxpar): Converts the (mostly scalar) physical parameters into vectors defined over the spatial discretization grid (e.g. eps_e, eps_s, dx, sigma_eff,

a_s), wraps scalar-or-function-handle parameters (k0_neg/pos, kappa, De, dlnfdce, Ds_pos, Ds_neg, i0sei, dU_dT_neg/pos) into function handles so that downstream code can treat them uniformly, computes the maximum solid-phase concentrations cs_max_neg/cs_max_pos from the nominal capacity Cbat and the stoichiometric window, calls deter_equil to build the equilibrium-potential curves, and, if ageing is enabled, computes the aged capacity and stoichiometric window consistent with the specified initial charge loss (via fcn_Q or fcn_Q_inv).

`deter_equil(p)`: Builds the equilibrium-potential functions U_neg, U_pos (and their derivatives dU_neg, dU_pos with respect to stoichiometry, used in the Jacobian) as fast look-up-table-based interpolants. If the user specified an electromotive-force curve EMF instead of (or in addition to) U_pos (p.equil_mode = 1), this function reconstructs consistent U_neg/U_pos curves from EMF and the given U_neg.

`fcn_system_matrices(stoichn, p, m, T)`: Assembles the sparse system matrices of the semi-analytical discretization that remain constant for a fixed time step dt: the solid-phase diffusion operators (Acs_hat, Bcs_hat), the solid-phase potential operator (Aphis, Bphis, Cphis), the electrolyte-phase flux-injection operator (Bce_hat, Bphie), and the surface/boundary extraction operators (Acs_bar, Ace_bar, Aphie_bar). It is called once during initialization and again, inside the time-stepping loop, whenever the step size dt changes. Depending on the chosen simplification for each concentration-dependent transport parameter, it also triggers a one-off call to De_matrices, Ds_matrices, kappa_matrices and/or nu_matrices.

`fcn_system_matrices2(p, m, cs_prevt, ce_prevt, T)`: Updates the subset of matrices that depend on the concentration at the *previous* time step, for any transport parameter using the [S2-I] simplification (which is otherwise re-evaluated at every Newton iteration under [S2-0], or never re-evaluated under [S2-II]). Called once per time step, before the Newton iteration begins.

`init(p, m, init_cond)`: Computes the initial state (cs, ce, phis, T, Closs, Rf) at $t = 0$. If init_cond is a struct, its fields are used directly. Otherwise, a steady-state solid-phase concentration profile is constructed from the requested initial SOC (inverting the equilibrium-potential curve first if an initial voltage was given instead), the electrolyte concentration is set to the uniform value ce0, and the initial solid-phase potential is set to the corresponding equilibrium potential. If ageing is enabled and a two-element init_cond was given, the initial charge loss is derived from the specified aged capacity via fcn_Q_inv.

`fcn_F(phis, ce_prevt, cs_prevt, FacLPL, T, i_app, p, m)`: Evaluates the nonlinear residual $F(\phi_s)$ of the coupled algebraic equations for a candidate solid-phase potential ϕ_s, together with every intermediate quantity needed to do so: the pore-wall flux jn (from ϕ_s), the electrolyte concentration ce and potential phie (both affine in ϕ_s and $\log(ce)$ through the precomputed matrices), the ageing side-reaction fluxes (jsei for SEI formation, jlpl for lithium plating, j_ox for cathode oxidation, each computed from its own Butler-Volmer-type expression and overpotential, and gated by p.ageingMech.SEI/LPL/Oxidation), the net intercalation flux j1 (jn minus the side-reaction fluxes), the solid-phase concentration cs (via superposition of the precomputed diffusion matrices with j1), the exchange current density i0, the equilibrium potential U (evaluated at the particle-surface stoichiometry), the main-reaction overpotential eta, and finally the residual F_phis itself, either from the full or the linearized [S1] Butler-Volmer equation. This is the function evaluated at every Newton iteration.

`fcn_J(cs, ce, FacLPL, CeRat, jn, j1, i0, eta, eta2, eta_ox, T, p, m)`: Evaluates the Jacobian `J_phis` of `F_phis` with respect to ϕ_s, by chaining together the (mostly matrix, i.e. linear) sensitivities of `ce`, `cs`, `phie`, the exchange current density, the equilibrium potential, and, if ageing is enabled, the SEI-formation, lithium-plating and oxidation fluxes, with respect to ϕ_s. Used together with `F_phis` to form the Newton update $\phi_s \leftarrow \phi_s - J^{-1}F$.

`fcn_T(jn, U, stoich, V, i_app, T_prevt, p)`: Advances the lumped thermal energy balance by one time step: convective heat loss to the ambient (using `hc`) plus the sum of reversible (entropic, using `dU_dT_neg/pos`) and irreversible heat generation, divided by the thermal mass `Cp` · `m`.

`compute_soc(cs, p)`: Computes the cell state-of-charge from the current solid-phase concentration by numerically integrating the (volume-averaged) particle concentration in the negative electrode over the particle radius, and mapping the result onto $[0, 1]$ using the fresh-cell stoichiometric window `s0_neg`/`s100_neg`.

`fcn_Q(Closs, stoich_Q_prev, indicator, p)`: Given a cumulative charge loss `Closs`, solves (via Newton's method, in sub-steps of at most 1000 C for robustness) for the stoichiometric window (`s0_neg`, `s100_neg`, `s0_pos`, `s100_pos`) and the resulting aged capacity `Q`, using the definitions of the maximum/minimum cell voltage (`OCV_max`, `OCV_min`) and lithium-inventory conservation. Called periodically (every 500 time steps) during an ageing simulation, and once during initialization.

`fcn_Q_inv(Q, stoich_Q_prev, indicator, p)`: The inverse of `fcn_Q`: given a desired aged capacity `Q`, solves for the cumulative charge loss `Closs` (and the associated stoichiometric window) that would produce it. Used to translate a user-specified aged capacity (via a two-element `init_cond`, or via `Closs_init` indirectly) into the internal charge-loss bookkeeping variable used by the rest of the simulation.

`fcn_Acs_j(nr)`: Builds the (dimensionless) finite-difference Laplacian-type operator, in spherical coordinates, for diffusion along the radial direction of a particle discretized into `nr` nodes. Used by `fcn_system_matrices` to build the full solid-phase diffusion operator for the negative and positive electrode particles.

`fcn_Aw(alpha1, alpha0, dx, p)`: Builds a generic finite-volume-method (FVM) operator for a 1-D transport equation across the cell width, given a (possibly node-dependent) transport coefficient `alpha1`, a scaling vector `alpha0`, and the node widths `dx`. The face values of `alpha1` are computed as a harmonic mean, a linear variation, or a weighted mean, depending on `p.fvm_method`. This is the common building block used to construct the solid-phase potential operator (`Aphis`), and, via the functions below, the electrolyte concentration, electrolyte potential, and solid-phase diffusion operators.

`De_matrices(ce, T, p, m)`: Builds the electrolyte-phase diffusion operator `Ace` (via `fcn_Aw`, using the electrolyte diffusion coefficient `De_eff`) and the matrices that express the electrolyte concentration `ce` as an affine function of ϕ_s and the applied current (`Gamma_ce`, `Phi_ce`), plus their surface/boundary-extracted counterparts. Called from `fcn_system_matrices`, `fcn_system_matrices2`, or `fcn_F`, depending on whether simplification [S2-II], [S2-I], or [S2-0] is selected for `De`.

`Ds_matrices(stoichn, stoichp, T, p, m)`: Builds the solid-phase diffusion operator (either the full radial finite-difference operator, or the 2-parameter polynomial approximation used under simplification [S3]) and the matrices expressing the solid-phase concentration `cs` as an affine function of ϕ_s and the applied current. Called

analogously to `De_matrices`, but for `Ds_pos` (`Ds_neg` is currently always evaluated directly, i.e. not subject to the [S2] simplification choice).

`kappa_matrices(ce, T, p, m)`: Builds the electrolyte ionic-conductivity operator `Aphie` (via `fcn_Aw`, using `kappa_eff`) and the matrices expressing the electrolyte potential `phie` as an affine function of ϕ_s, the applied current, and $\log(ce)$. Called analogously to `De_matrices`, for the parameter `kappa`.

`nu_matrices(ce, T, p, m)`: Builds the matrix `Pi_phie` relating the diffusional-conductivity term (dependent on `dlnfdce`, the mean molar activity coefficient) to $\log(ce)$ in the electrolyte-potential equation. Called analogously to `De_matrices`, for the parameter `dlnfdce`.

`qinterp1(x, Y, xi, methodflag)`: A fast nearest-neighbour/linear interpolation routine for evenly-spaced `x`, used internally by the equilibrium-potential functions returned from `deter_equil` (`U_pos`, `U_neg`, `dU_pos`, `dU_neg`) since these are evaluated at every Newton iteration and `interp1` would be comparatively slow.

`dispstat(TXT, varargin)`, `print_loop(...)`: Console-output helper functions used to print (and, depending on `p.verbose`, overwrite in place) simulation progress, used only for user feedback and not part of the numerical procedure.

5 User-Configurable Parameters

The user-configurable parameters are given as an (optional) input to the `DFN` function in a `struct`. The fields that can be specified in this `struct` are given as follows.

5.1 Simulation Parameters

`dt`: [s] the step size. Default value is 1.

`grid`: is a struct where each of its fields specify the coarseness of the grid parameters. Note that the grid parameters can also be changed (more effectively) through `set_grid`, however setting the simplifications in `grid` is more intuitive. The fields in `grid` are:

- `nn` [-] number of FVM nodes for the negative electrode. Default value is 10.
- `ns` [-] number of FVM nodes for the separator. Default value is 10.
- `np` [-] number of FVM nodes for the positive electrode. Default value is 10.
- `nrn` [-] number of FDM nodes for the negative electrode along the radial direction. Default value is 10.
- `nrp` [-] number of FDM nodes for the positive electrode along the radial direction. Default value is 10.

`set_grid`: vector containing the grid parameters, of which its elements can be specified in the following order: `set_grid = [nn ns np nrn nrp]`. This field is especially useful when it is desired to change the set of grid parameters effectively. Note that the grid parameters can be changed either through this parameter, or through the above parameters, seperately. If both `set_grid` and either of the other parameters are given, then the values specified in `set_grid` always take precedence. For an overview of how to effectively make a trade-off between model accuracy and computation time using the grid parameters, please see [2] and Section 2.3. Default value is `[10 10 10 10 10]`.

`tol`: [-] is the tolerance used to terminate the Newton's algorithm for solving the set of algebraic equations. Default value is 1e-3.

`iter_max`: [-] specifies the maximum number of iterations that the Newton's method at each time step may take, before the step size is reduced and the step retried. Default value is 100.

`Vmin`: [V] specifies the minimum allowable terminal voltage. Exceeding this minimum voltage terminates the simulation. Default value is 2.5.

`Vmax`: [V] specifies the maximum allowable terminal voltage. Exceeding this maximum voltage terminates the simulation. Default value is 6.

`verbose`: specifies the level of verbosity for the simulation: 0 for no output, 1 to only indicate the start/end of the simulation with computation time, 2 to additionally show the applied current and voltage while simulating. Default value is 2.

`current_interp`: specifies how the applied current (and temperature, if given) should be interpolated when `input_current` is given by an array. Can be chosen as any of the methods specified in the documentation of the MATLAB `griddedInterpolant` function. Default value is `'linear'`.

`current_extrap`: specifies how the applied current should be extrapolated when `input_current` is given by an array. Can be chosen as any of the methods specified in the documentation of the MATLAB `griddedInterpolant` function. Default value is `'linear'`.

`fvm_method`: specifies how the face values of the FVM discretization are computed. Can be chosen as either 1,2,3, where 1 specifies the harmonic mean, 2 specifies assuming a linear variation, and 3 specifies the weighted mean. Default value is 1.

`thermal_dynamics`: toggles the lumped thermal model. When specified as 1 thermal dynamics are calculated (the cell temperature is then given by `fcn_T`), when specified as 0 thermal dynamics are ignored (the cell is held at `T_amb`, or at a supplied temperature profile). Default value is 0.

`T_amb`: [K] specifies the ambient temperature. Default value is 298.15.

`Closs_init`: [C] the initial condition for the cumulative charge lost due to side reactions, used when `init_cond` is not given as a struct or as a two-element vector. Default value is 0.

`ageing`: toggles ageing. When specified as 1 ageing is enabled, when specified as 0 ageing is disabled. Default value is 0.

`ageingMech`: a struct with the fields `SEI`, `LPL`, `Oxidation` and `LimitedSEI`, each toggling (1/0) whether the corresponding mechanism is active when ageing is enabled. All default to 0; see Section 5.2.3.

- `SEI` enables the SEI-formation side reaction at the negative electrode.
- `LPL` enables the lithium-plating side reaction at the negative electrode.
- `Oxidation` enables the oxidation side reaction at the positive electrode.
- `LimitedSEI` toggles whether SEI growth is limited by a film-resistance-dependent factor (1) or proceeds unrestricted (0).

`simp`: is a struct where each of its fields specify the type of simplifications that the user wants to make. Note that the simplifications can also be changed (more effectively) through `set_simp`, however setting the simplifications in `simp` is more intuitive. The fields in `simp` are:

- `kappa` specifies the type of simplification to make on the parameter `kappa`, where `0` indicates no simplification, `1` indicates that [S2-I] is applied and `2` indicates that [S2-II] is applied. Default value is 1.
- `De` specifies the type of simplification to make on the parameter `De`, where `0` indicates no simplification, `1` indicates that [S2-I] is applied and `2` indicates that [S2-II] is applied. Default value is 1.
- `dlnfdce` specifies the type of simplification to make on the parameter `dlnfdce`, where `0` indicates no simplification, `1` indicates that [S2-I] is applied and `2` indicates that [S2-II] is applied. Default value is 1.
- `Ds_pos` specifies the type of simplification to make on the parameter `Ds_pos`, where `0` indicates no simplification, `1` indicates that [S2-I] is applied and `2` indicates that [S2-II] is applied. Default value is 1.
- `butler_volmer` specifies the type of simplification to make on the Butler-Volmer equation, where `0` indicates no simplification, and `1` indicates that [S1] is applied. Default value is 0.
- `solid_phase_diffusion` specifies the type of simplification to make on the solid-phase diffusion equation, where `0` indicates no simplification, and `1` indicates that [S3] is applied. Default value is 0.

For an overview of how to effectively make a trade-off between model accuracy and computation time using these simplifications, please see [2] and Section 2.2

`set_simp`: This variable is a vector of 6 integers of the form `set_simp = [v1 v2 v3 v4 v5 v6]`, where `v1`, `v2`, `v3`, `v4`, `v5`, and `v6` relate to the simplifications made to `kappa`, `De`, `dlnfdce`, `Ds_pos`, the Butler-Volmer equation, and the solid-phase diffusion dynamics, respectively. The values of the elements are chosen in the same way as described in `simp`. Note that if both `set_simp` and any of the fields in `simp` are given, then the values specified in `set_simp` always take precedence. Default value is `[2 2 2 2 1 0]`.

5.2 Material-Specific Parameters

5.2.1 DFN Model Parameters

`F`: [C/mol] Faraday's constant

`R`: [J/mol/K] Ideal gas constant

`delta_neg`: [m] Negative electrode thickness

`delta_sep`: [m] Separator thickness

`delta_pos`: [m] Positive electrode thickness

`R_neg`: [m] Particle radius in the negative electrode

`R_pos`: [m] Particle radius in the positive electrode

`epse_neg`: [-] Electrolyte volume fraction at the negative electrode

`epse_sep`: [-] Electrolyte volume fraction at the separator

`epse_pos`: [-] Electrolyte volume fraction at the positive electrode

`epss_neg`: [-] Active material volume fraction at the negative electrode

`epss_pos`: [-] Active material volume fraction at the positive electrode

`p_neg`: [-] Bruggeman porosity exponent at the negative electrode

`p_sep`: [-] Bruggeman porosity exponent at the separator

`p_pos`: [-] Bruggeman porosity exponent at the positive electrode

`Ds_neg`: [m^2/s] Solid-phase Li diffusion coefficient at the negative electrode. Can be specified either as a constant value or an anonymous function with temperature as input.

`Ds_pos`: [m^2/s] Solid-phase Li diffusion coefficient at the positive electrode. Can be specified either as a constant value or an anonymous function with stoichiometry and temperature as inputs.

`De`: [m^2/s] Electrolyte-phase Li diffusion coefficient. Can be specified either as a constant value or an anonymous function with concentration and temperature as inputs.

`dlnfdce`: [-] Mean molar activity coefficient. Can be specified either as a constant value or an anonymous function with concentration and temperature as inputs.

`kappa`: [S/m] Ionic conductivity. Can be specified either as a scalar value or an anonymous function with concentration and temperature as inputs.

`sigma_neg`: [S/m] Electronic conductivity of the negative electrode.

`sigma_pos`: [S/m] Electronic conductivity of the positive electrode.

`Rf_neg`: [Ω/m^2] SEI film resistance in the negative electrode. Can be specified either as a scalar value or a vector value indicating the resistance value at each spatial node in the negative electrode.

`Rf_pos`: [Ω/m^2] SEI film resistance in the positive electrode.

`t_plus`: [-] Transference number of Li-ions.

`A_surf`: [m^2] Electrode plate surface area.

`R_cc`: [Ω/m^2] Overall current collector resistance. Default value is 0.

`alpha_a`: [-] Anodic charge-transfer coefficient. Note that the cathodic charge-transfer coefficient `alpha_c` is computed from `alpha_a` as `alpha_c = 1-alpha_a`.

`k0_neg`: [$mol^{-3/2}m^{-1/2}s^{-1}$] Kinetic constant in the negative electrode. Can be specified either as a constant value or an anonymous function with temperature as input.

`k0_pos`: [$mol^{-3/2}m^{-1/2}s^{-1}$] Kinetic constant in the positive electrode. Can be specified either as a constant value or an anonymous function with temperature as input.

`ce0`: [mol/m^3] Average (initial) electrolyte concentration.

`Cbat`: [C] Reversible (fresh) capacity of the battery.

`s0_neg`: [-] Stoichiometry at 0% state-of-charge in the negative electrode of a fresh cell.

`s100_neg`: [-] Stoichiometry at 100% state-of-charge in the negative electrode of a fresh cell.

`s0_pos`: [-] Stoichiometry at 0% state-of-charge in the positive electrode of a fresh cell.

`s100_pos`: [-] Stoichiometry at 100% state-of-charge in the positive electrode of a fresh cell.

`U_neg`: [V] Negative electrode equilibrium potential, given by an anonymous function, with the stoichiometry as the input and equilibrium potential the output.

`U_pos`: [V] Positive electrode equilibrium potential, given by an anonymous function, with the stoichiometry as the input and equilibrium potential the output.

`EMF`: [V] Electro-motive force of the cell, given by an anonymous function, with cell state-of-charge as the input. Note that this field is optional and when this field is provided in `param`, the toolbox will automatically compute a corresponding `U_pos`, based on either the default `U_neg` or a specified `U_neg`, via `deter_equil`.

5.2.2 Thermal Model Parameters

`dU_dT_neg`: [V/K] Entropy change in the negative electrode, given by either by a scalar or an anonymous function, with the stoichiometry as the input.

`dU_dT_pos`: [V/K] Entropy change in the positive electrode, given by either by a scalar or an anonymous function, with the stoichiometry as the input.

`m`: [kg] Mass of the cell.

`hc`: [W/K] Convective heat transfer coefficient.

`Cp`: [J/kg/K] Specific heat capacity of the cell.

5.2.3 Ageing Model Parameters

The ageing model implemented in TOOFAB accounts for three side reactions that consume cyclable lithium and/or degrade transport properties: SEI (solid-electrolyte interphase) formation at the negative electrode, lithium plating at the negative electrode, and an oxidation side reaction at the positive electrode. Each mechanism can be enabled independently through `ageingMech.SEI`, `ageingMech.LPL` and `ageingMech.Oxidation`, respectively; the parameters below only have an effect for a mechanism that is enabled. The resulting fluxes and their effect on the cell state (film resistance, porosity, capacity fade) are described in Section 4.2. The precise definition of the over-potential driving the SEI-formation reaction depends on where SEI formation is assumed to take place; the default implementation follows the assumption used by [8].

`i0sei`: [A/m^2] Exchange current density of the SEI-formation side reaction. Can be specified either as a scalar or an anonymous function of temperature. Default value is 0.016.

`ac_sei`: [-] Cathodic charge-transfer coefficient of the SEI-formation side reaction. Default value is 0.5.

`lamda_sei`: [-] Resistance-limiting factor for SEI formation, used as $\exp(-\texttt{Rf} \cdot \texttt{lamda_sei})$ when `ageingMech.LimitedSEI` is enabled (a larger value makes the SEI-formation flux decay faster as the film resistance `Rf` builds up). Default value is 15.

`B1_s`: [-] Coefficient relating the SEI-formation flux to the growth rate of the SEI film resistance `Rf`. Default value is 25.

`B2_s`: [-] Coefficient relating the SEI-formation flux to the loss rate of negative-electrode porosity `epse_neg`. Default value is -5.

`i0lpl`: [A/m^2] Exchange current density of the lithium-plating side reaction. Default value is 4×10^{-4}.

`ac_lpl`: [-] Cathodic charge-transfer coefficient of the lithium-plating side reaction. The anodic charge-transfer coefficient of the same reaction is computed as $1-$`ac_lpl`, analogous to `alpha_a`/`alpha_c` for the main reaction. Default value is 0.7.

`B1_l`: [-] Coefficient relating the lithium-plating flux to the growth rate of the SEI film resistance `Rf` (plated lithium is assumed to contribute to the resistive film in the same way as SEI). Default value is 15.

`B2_l`: [-] Coefficient relating the lithium-plating flux to the loss rate of negative-electrode porosity `epse_neg`. Default value is -5.

`i0ox`: [A/m^2] Exchange current density of the positive-electrode oxidation side reaction. Default value is 7×10^{-42}.

`aa_ox`: [-] Anodic charge-transfer coefficient of the positive-electrode oxidation side reaction. Default value is 0.5.

`B0`: [-] Coefficient relating the oxidation flux to the loss rate of positive-electrode active-material volume fraction `epss_pos`. Default value is 5.

`TotCycle`: [-] Total number of CC-CV cycles used for bookkeeping in the `mem` struct during a closed-loop, multi-cycle simulation. Default value is 1.

6 Parameter Determination

6.1 Overview

Many of the physical parameters of the DFN model (Section 5) cannot be measured directly and must instead be estimated from terminal-voltage (and, optionally, temperature) measurements of a cell subjected to a known current profile. `parameter_determination.m` implements a workflow for doing so. Rather than fitting the physical parameters directly, the function internally works with a smaller set of dimensionless, lumped parameters (referred to as *hat-parameters*, denoted `ph` in the code) that combine several physical quantities (e.g., a diffusion coefficient together with a particle radius) into groups that are better identifiable from voltage data alone; `phat2p` and `p2phat` convert between this lumped representation and the full physical parameter struct `p` used by `DFN`. The parameter estimation procedure relates directly to the work of [9] and the extension of [10].

Because not every hat-parameter is equally identifiable from a given data set, the estimation workflow first ranks the parameters by how much they influence the simulated output (a one-at-a-time sensitivity analysis, Section 6.4), and then estimates only the most influential subset with bounded nonlinear least-squares, leaving the remaining, poorly-identifiable parameters at their nominal (mid-range) values. This mirrors the general workflow described for the DFN model simplifications: a deliberate trade-off, here between the number of free parameters and the risk of over-fitting a small dataset.

6.2 parameter_determination

`parameter_determination`: The `parameter_determination` function is the main, user-facing function for estimating DFN model parameters from experimental data. Section 6.4 describes what it does step by step, and Section 6.5 documents its internal (local) functions.

6.2.1 Syntax

```
[p, ph, results] = parameter_determination(equil_data, est_data, options)
```

6.2.2 Input Arguments

`equil_data`: A `struct` containing information about the equilibrium (open-circuit) behaviour of the cell, together with its reference capacity. It must always contain the field

- `Cbat` [C] the (approximate) fresh capacity of the cell, used to scale the parameter bounds in `determine_parameter_ranges` and the free-choice geometry in `phat2p`.

and, in addition, either

- `EMF` [V], an anonymous function of the cell state-of-charge giving the cell's electro-motive force (mirroring `EMF` in Section 5). In this mode (`equil_mode = 1`), the stoichiometric window (`s0_neg`, `s0_pos`, `s100_neg`, `s100_pos`) is treated as unknown and included among the estimated parameters, since it cannot be uniquely determined from the EMF curve alone.

or

- `U_neg`, `U_pos` [V], anonymous functions of stoichiometry giving the negative/positive electrode equilibrium potentials (as in Section 5), together with the fixed stoichiometric window `s0_neg`, `s0_pos`, `s100_neg`, `s100_pos`. In this mode (`equil_mode = 0`), the stoichiometric window is taken as known and is not re-estimated.

`est_data`: The experimental data to fit. Can be given either as a single `struct`, or as a cell array of structs when several experiments (e.g., different C-rates or temperatures) should be fitted simultaneously. Each struct must contain the fields

- `t` [s] the measurement time vector.
- `I` [A] the applied current at each time sample (same sign convention as `input_current`).
- `V` [V] the measured terminal voltage at each time sample.

and, optionally,

- `T` [K] the measured cell temperature at each time sample, required when `options.T_objective` is enabled.

It is recommended that the data be resampled to a uniform 1 s sample time before use.

`options`: An optional `struct` used to configure the estimation routine; in most cases only `options.dfn.T_amb` needs to be set. Any field not specified is assigned the default value listed below.

- `sensitivity_analysis` toggles whether the sensitivity-ranking step (Section 6.4) is performed. Default value is 1.
- `parameter_estimation` toggles whether the bounded nonlinear least-squares fit is performed. Default value is 1.
- `sens_perturb` [-] the perturbation size, in normalized parameter units, used for the one-at-a-time finite-difference sensitivity analysis. Default value is 0.1.
- `sens_perturb_mode` selects the finite-difference scheme used for the sensitivity analysis: `-1` perturbs only downward, `1` only upward, and `0` averages both directions. Default value is 1.
- `plot_sensitivity` toggles whether a bar plot of the ranked sensitivity magnitudes is generated. Default value is 1.
- `par_names_plot` a cell array of LaTeX-formatted parameter labels, in the same order as the internal parameter list, used to annotate the sensitivity plot.
- `beta_0` [-] the nominal normalized parameter value (in $[0, 1]$) used as the sensitivity-analysis reference point and, unless overridden by `rand_init` or `input_par`, as the optimizer's initial guess. Default value is 0.5, i.e., the middle of each parameter's allowed range.
- `rand_init` toggles using a random initial guess for the optimizer instead of `beta_0`. Default value is 0.
- `range_scaling` [-] widens the interval from which the random initial guess is drawn, from $[0, 1]$ to $[-$`range_scaling`, $1+$`range_scaling`$]$, when `rand_init` is enabled. Default value is 0.
- `n_estimation` [-] the number of most-sensitive parameters to actually estimate. If left at its default value of 0, the toolbox prompts for this number interactively in the command window once the sensitivity ranking is available.
- `use_input_par_as_init` toggles whether, when `input_par` is given, it is used (converted to normalized form) as the optimizer's initial guess instead of `beta_0`/`rand_init`. Default value is 1.
- `V_objective` toggles whether the terminal voltage is included in the sensitivity/estimation objective. Default value is 1.
- `T_objective` toggles whether the cell temperature is included in the sensitivity/estimation objective (and, if enabled, forces `thermal_dynamics` on for the internal simulations). Default value is 0.
- `opt_tolX` [-] the step-size tolerance passed to `lsqnonlin`. Default value is 10^{-4}.
- `input_par` an optional full physical DFN parameter `struct` (as in Section 5) used to seed the estimation, via `p2phat`.
- `est_par`, `est_methods` a manually-specified list of parameter names (and their linear-vs-logarithmic mapping methods) to estimate, used instead of the sensitivity-ranked list when `sensitivity_analysis` is disabled.
- `Caged` [C] an optional aged capacity. If given, the internal forward simulations are run with ageing enabled, using the two-element form of `init_cond` (Section 4.1) to reproduce the specified aged capacity.

- `dfn` a `struct` of `DFN` simulation-parameter overrides (Section 5.1), merged into every internal call to `DFN`. Defaults to `thermal_dynamics` = 0, `set_simp` = `[2 2 2 2 1 0]`, `T_amb` = 299.15 K, `verbose` = 0, `set_grid` = `[10 10 10 10 10]`.

6.2.3 Output Arguments

`p`: The estimated (or, if `parameter_estimation` is disabled, nominal) full physical DFN parameter `struct`, directly usable as the `param` argument to `DFN`.

`ph`: The same estimated parameters, expressed in the internal normalized/lumped hat-parameter representation. Useful for inspecting the identified dimensionless groups, or for passing to a follow-up call as `options.input_par` (after conversion back with `phat2p`, since `input_par` expects the physical representation).

`results`: A `struct` collecting diagnostic information from the estimation run.

- `sensitivity` (present when `sensitivity_analysis` is enabled): a `struct` with fields `Pi_init` and `Pi_ranked` (the original and ranked parameter indices), `ranked_parnames` (the parameter names, most-to-least sensitive), `sensitivity_magnitude` (the normalized sensitivity magnitude of each ranked parameter), and `ranked_methods` (the linear/logarithmic mapping method of each ranked parameter).
- `fval`, `beta_est`, `beta_0`, `est_time`, `lsqnonlin_output` (present when `parameter_estimation` is enabled): the final residual, the estimated and initial normalized parameter vectors, the wall-clock estimation time, and the full diagnostic output struct returned by MATLAB's `lsqnonlin`, respectively.

6.3 Example

`paramEst_DFN_Example.mlx` : A MATLAB live script that walks through parameter estimation of the DFN model interactively, including setting up of the equil struct, which contains the structed EMF curve required for parameter estimation.

6.4 Procedure: How `parameter_determination.m` Works

1. **Merge options.** The default `options` (including the nested `options.dfn`) are overwritten with any user-supplied values.
2. **Seed from a supplied parameter set.** If `options.input_par` was given, it is run through one trivial simulation just to obtain a fully-expanded parameter `struct`, which is then converted to the hat-parameter representation via `p2phat` for later use as the optimizer's initial guess.
3. **Determine the estimable parameter list and bounds.** The list of hat-parameters that may be estimated (`par_names`) depends on `equil_mode`: if an `EMF` curve was supplied in `equil_data`, the stoichiometric window is added to the list. `determine_parameter_ranges` then supplies a [min, max] bound, scaled by `Cbat` where appropriate, for every parameter in the list.
4. **Select the normalization method.** For each parameter, `estimate_parameters` chooses whether the normalized value $\beta \in [0, 1]$ should be mapped

to the physical range linearly or logarithmically (`beta2par`): logarithmic mapping is used whenever the bounds span more than a decade, linear otherwise.

5. **Sensitivity analysis** (only if `sensitivity_analysis`). `sensitivity_analysis` perturbs each normalized parameter individually, up and/or down by `sens_perturb` around the nominal value `beta_0` (in parallel, one simulation per parameter and perturbation direction, via `dfn_model`), and finite-differences the resulting change in the simulated output to build a sensitivity matrix. This matrix is factorized with a column-pivoted QR decomposition; the resulting pivot order ranks the parameters by their (orthogonalized) contribution to the model output, and the diagonal of the triangular factor gives each parameter's normalized sensitivity magnitude. If `plot_sensitivity` is enabled, a bar plot of the ranked magnitudes is produced.

6. **Select which parameters to estimate.** If a sensitivity ranking is available, the top `n_estimation` parameters are selected (prompting for this number interactively if it was left at its default of 0); otherwise, the manually-specified `est_par`/`est_methods` are used directly.

7. **Parameter estimation** (only if `parameter_estimation`). An initial normalized guess is built from `beta_0` (or a random draw if `rand_init` is enabled, or from `options.input_par` via `parvar_inv` if `use_input_par_as_init` is enabled). MATLAB's `lsqnonlin` then minimizes, over the normalized parameters bounded to $[0, 1]^n$, the residual returned by `evaluate_objective`: for every supplied experiment in `est_data`, the DFN model is resimulated at the trial parameters (via `dfn_model`) and the scaled difference between the simulated and measured voltage and/or temperature (as selected by `V_objective`/`T_objective`) is concatenated across experiments into the residual vector.

8. **Assemble the output.** The estimated (or, if estimation was skipped, nominal) normalized parameters are converted to the hat-parameter `struct ph`, then to the full physical parameter `struct p` via `phat2p`, the equilibrium-potential and stoichiometric-window fields are attached from `equil_data` (or from the estimated stoichiometric window, in EMF mode), and the `options.dfn` simulation overrides are merged in, giving a `p` that can be passed directly to `DFN`.

6.5 Description of Internal (Local) Functions

All functions described in this section are local functions defined within `parameter_determination.m`.

`estimate_parameters(model, y_exp, equil_data, par_names, ranges, est_data, options)`:

Orchestrates steps 4–7 of Section 6.4: selects the linear/logarithmic normalization method for each parameter, optionally runs `sensitivity_analysis`, selects the parameter subset to estimate, and, if enabled, runs the bounded `lsqnonlin` fit against `evaluate_objective`.

`evaluate_objective(beta, model, par, y_exp, est_data, equil_data, est_par, est_methods, ranges, options)`:

The residual function minimized by `lsqnonlin`. Converts the trial normalized parameters `beta` to physical values (via `beta2par`), rebuilds the equilibrium potentials (via `determine_equilibrium_potentials`), resimulates the model for every dataset in `est_data`, and returns the concatenated, scaled voltage/temperature residual. Simulations that fail to

produce a long-enough or finite output are replaced by a zero-residual fallback so that a single non-converging trial parameter set does not crash the optimizer.

`beta2par(beta, par_names, ranges, method)`: Maps a normalized parameter vector `beta` (each entry in $[0, 1]$) to physical hat-parameter values, using either linear or log-linear interpolation between `ranges.<name>_min` and `ranges.<name>_max`, per the method flag selected in `estimate_parameters`.

`parvar_inv(theta, par_min, par_max, method)`: The inverse of `beta2par`: maps a physical value back to its normalized $\beta \in [0, 1]$ representation. Used to convert `options.input_par` into an initial guess for the optimizer.

`determine_equilibrium_potentials(equil_data, par, options)`: Builds the equilibrium-potential interpolants `U_neg`/`U_pos` for a given trial parameter set, mirroring `deter_equil` in `DFN.m`: in EMF mode it reconstructs `U_pos` from the supplied `EMF` curve and a fixed reference `U_neg`, otherwise it uses the supplied `U_neg`/`U_pos` directly. Called at every sensitivity-analysis perturbation and at every optimizer iteration, since the equilibrium potentials depend on the (possibly trial) stoichiometric window.

`sensitivity_analysis(model, equil_data, par_names, ranges, methods, input_data, options)`: Implements the one-at-a-time perturbation and QR-pivoting-based ranking procedure described in step 5 of Section 6.4, and produces the ranked-sensitivity bar plot when requested.

`dfn_model(equil, data, ph, options)`: The forward-model wrapper called by both `sensitivity_analysis` and `evaluate_objective`. Converts the trial hat-parameters to a physical parameter struct (via `phat2p`), merges in the `options.dfn` overrides, enables ageing with the aged capacity `Caged` if specified, calls `DFN` with the experiment's current profile (and measured temperature profile, if available and ageing is not being used to set the initial condition), interpolates the simulated output onto the experimental time samples, and returns the voltage and/or temperature signal selected by `V_objective`/`T_objective`.

`phat2p(ph, Cbat)`: Converts the normalized/lumped hat-parameter `struct ph` into a full physical DFN parameter `struct p`. A small set of "free-choice" geometric parameters (electrode thicknesses, particle radii, porosities, and a nominal capacity-per-area used to size `A_surf`) are fixed to representative literature values, and the remaining physical parameters (diffusion coefficients, conductivities, transference number, kinetic constants, film resistance, current-collector resistance, etc.) are analytically backsolved from the dimensionless hat-parameter groups, including, where applicable, an Arrhenius temperature dependence built from the fitted activation-energy groups (`knEa`, `kpEa`, `DEa`).

`p2phat(p)`: The inverse of `phat2p`: converts a full physical DFN parameter `struct` into the normalized/lumped hat-parameter representation. Used to convert a user-supplied `options.input_par` into a form that can seed the estimation.

`determine_parameter_ranges(Cbat)`: Returns the `struct ranges`, containing literature/experience-based [min, max] bounds (fields `<name>_min`/`<name>_max`) for every hat-parameter that can be estimated, most of them scaled by the given `Cbat`.

Acknowledgements

`BPX_to_TOOFAB.m`, included with the `Basic_DFN_Example_livescript` example, is the result of a collaboration between TNO and the Control

Systems group of the Department of Electrical Engineering, Eindhoven University of Technology [11].

Index

A_surf, 18
aa_ox, 20
ac_lpl, 20
ac_sei, 19
ageing, 16
ageingMech, 16
alpha_a, 18

B1_l, 20
B1_s, 19
B2_l, 20
B2_s, 20
beta_0, 22
BO, 20

Caged, 22
Cbat, 18
ce0, 18
Closs_init, 16
Cp, 19
current_extrap, 16
current_interp, 16

De, 18
delta_neg, 17
delta_pos, 17
delta_sep, 17
DFN, 6
dfn, 23
dlnfdce, 18
Ds_neg, 18
Ds_pos, 18
dt, 15
dU_dT_neg, 19
dU_dT_pos, 19

EMF, 19
epse_neg, 17
epse_pos, 17
epse_sep, 17
epss_neg, 18
epss_pos, 18
equil_data, 21
est_data, 21
est_methods, 22
est_par, 22

F, 17
fvm_method, 16

grid, 15

hc, 19

i0lpl, 20
i0ox, 20
i0sei, 19
init_cond, 7
input_current, 6
input_par, 22
iter_max, 16

k0_neg, 18
k0_pos, 18
kappa, 18

lamda_sei, 19
LimitedSEI, 16

m, 19

n_estimation, 22

opt_tolX, 22
options, 21
out, 8

p, 23
p_neg, 18
p_pos, 18
p_sep, 18
par_names_plot, 22
param, 8
parameter_determination, 21
parameter_estimation, 22
ph, 23
plot_sensitivity, 22

R, 17
R_cc, 18
R_neg, 17
R_pos, 17
rand_init, 22
range_scaling, 22
results, 23
Rf_neg, 18
Rf_pos, 18

s0_neg, 18
s0_pos, 18
s100_neg, 18
s100_pos, 19
sens_perturb, 22
sens_perturb_mode, 22
sensitivity_analysis, 22
set_grid, 15
set_simp, 17
sigma_neg, 18
sigma_pos, 18
simp, 17

T_amb, 16
T_objective, 22
t_plus, 18
tf, 7
thermal_dynamics, 16
tol, 15
TotCycle, 20
U_neg, 19
U_pos, 19
use_input_par_as_init, 22
V_objective, 22
verbose, 16
Vmax, 16
Vmin, 16

References

[1] M. Doyle, T. F. Fuller, J. Newman, Modeling of galvanostatic charge and discharge of the lithium/polymer/insertion cell, J. Electrochem. Soc. 140 (6) (1993) 1526–1533. doi:10.1149/1.2221597.

[2] Z. Khalik, M. C. F. Donkers, H. J. Bergveld, Model simplifications and their impact on computational complexity for an electrochemistry-based battery modeling toolbox, J. Power Sources (2021).

[3] J. Newman, Fortran programs for the simulation of electrochemical systems, http://www.cchem.berkeley.edu/jsngrp/fortran.html (accessed November 10, 2020).

[4] M. Torchio, L. Magni, R. B. Gopaluni, R. D. Braatz, D. M. Raimondo, LIONSIMBA: A matlab framework based on a finite volume model suitable for Li-ion battery design, simulation, and control, J. Electrochem. Soc. 163 (7) (2016) A1192–A1205. doi:10.1149/2.0291607jes.

[5] L. Cai, R. E. White, Mathematical modeling of a lithium ion battery with thermal effects in COMSOL Inc. Multiphysics (MP) software, J. Power Sources 196 (14) (2011) 5985–5989. doi:10.1016/j.jpowsour.2011.03.017.

[6] P. Ramadass, B. Haran, P. M. Gomadam, R. White, B. N. Popov, Development of First Principles Capacity Fade Model for Li-Ion Cells, Journal of The Electrochemical Society 151 (2) (2004) A196. doi:10.1149/1.1634273.

[7] L. Rao, J. Newman, Heat-Generation Rate and General Energy Balance for Insertion Battery Systems, J. Electrochem. Soc. 144 (8) (1997) 2697–2704. doi:10.1149/1.1837884.

[8] F. le Roux, T. Donkers, H. J. Bergveld, Lithium-ion battery ageing modeling: Towards physically consistent implementations of sei growth, lithium plating, and cathode oxidation, Journal of Power Sources 664 (2026) 238926. doi:https://doi.org/10.1016/j.jpowsour.2025.238926.
URL https://www.sciencedirect.com/science/article/pii/S0378775325027636

[9] Z. Khalik, M. Donkers, J. Sturm, H. J. Bergveld, Parameter estimation of the doyle–fuller–newman model for lithium-ion batteries by parameter normalization, grouping, and sensitivity analysis, Journal of Power Sources 499 (2021) 229901.

[10] F. A. le Roux, H. Bergveld, M. Donkers, Improved parameter estimation of the doyle-fuller-newman model by incorporating temperature dependence, IFAC-PapersOnLine

56 (2) (2023) 6136–6141, 22nd IFAC World Congress. doi:https://doi.org/10.1016/j.ifacol.2023.10.714.
URL https://www.sciencedirect.com/science/article/pii/S2405896323010911

[11] A. M. Diaz Aguilar, A. A. Lodge, F. S. Hoekstra, Unlocking electrochemical battery modelling: an open-source suite comparison methodology**this research has received funding from the european union's horizon 2020 research and innovation programme under grant agreement no 101103898 under title of nextbms (https://nextbms.eu)., IFAC-PapersOnLine 59 (5) (2025) 169–174, 11th IFAC Symposium on Advances in Automotive Control AAC 2025. doi:https://doi.org/10.1016/j.ifacol.2025.07.100.
URL https://www.sciencedirect.com/science/article/pii/S2405896325004537